

PENTEST & RAPPORTAGE

Mohammed Ouaali
Aliriza Sari
Damian Buskens

16-6-2026

Inhoud

1.	Pentest plan.....	2
1.1	Scope en aanpak.....	2
1.2	Focusgebieden	2
2.	Uitvoering.....	3
2.1	AO7:2021 Hardcoded AWS secrets	3
2.1.1	Oplossing.....	3
2.2	AO1:2021 Path traversal.....	3
2.3	AO9:2021 PII in applicatielogs.....	4
2.3.1	Oplossing.....	4
2.4	A01:2021 Path traversal in DefaultAttachmentHandler	4
2.4.1	Oplossing.....	5
2.5	A05/A08:2021 Upload allowlist bypass.....	6
2.5.1	Oplossing.....	7
2.6	A04/A05:2021 Malformed base64 upload.....	9
2.6.1	Oplossing.....	10
2.7	A01:2021 Download-autorisatie / IDOR.....	10
2.7.1	Oplossing.....	11

1. Pentest plan

1.1 Scope en aanpak

We hebben gekozen voor een gerichte white-box penetratie test. In plaats van blind te scannen gebruiken we de inzichten uit de eerdere gap-analyse en risicomatrix om gerichte aanvallen uit te voeren op de zwakste plekken van de applicatie. Hierbij hanteren we de methodiek van de OWASP top 10, 2021.

1.2 Focusgebieden

Op basis van de risicoscores richten we de test op de volgende punten:

1. **OWASP A07:2021 - Identification and Authentication Failures:** Extractie van hardcoded Cloud Credentials.
2. **OWASP A01:2021 - Broken Access Control:** Local File Inclusion (LFI) / Path Traversal via de REST API.
3. **OWASP A09:2021 - Security Logging and Monitoring Failures:** PII-extractie via applicatielogging.
4. **OWASP A01:2021 - Broken Access Control:** Path traversal in DefaultAttachmentHandler door onveilige combinatie van attachmentDir en fileName.
5. **OWASP A05:2021 - Security Misconfiguration / OWASP A08:2021 - Software and Data Integrity Failures:** Upload allowlist bypass door lege standaardconfiguratie voor toegestane bestandsextensies.
6. **OWASP A04:2021 - Insecure Design / OWASP A05:2021 - Security Misconfiguration:** Malformed base64 upload door onvoldoende validatie van base64Content.
7. **OWASP A01:2021 - Broken Access Control:** Download-autorisatie / IDOR, waarbij wordt gecontroleerd of een gebruiker attachment bytes kan downloaden van een patiënt waar hij geen toegang toe heeft.
8. **OWASP A01:2021 - Broken Access Control:** Patient/visit/encounter mismatch bij upload, waarbij wordt gecontroleerd of een attachment verkeerd aan een andere patiënt, visit of encounter gekoppeld kan worden.
9. **OWASP A09:2021 - Security Logging and Monitoring Failures:** Log injection en gevoelige data in upload- en delete-logs, waarbij wordt gecontroleerd of logs geen patiëntgegevens of vervalste logregels bevatten.
10. **OWASP A01:2021 - Broken Access Control:** Search metadata disclosure, waarbij wordt gecontroleerd of een gebruiker attachmentmetadata van een andere patiënt kan ophalen via search parameters.

2. Uitvoering

2.1 AO7:2021 Hardcoded AWS secrets

Momenteel is het zo dat in de bestand "AttachmentsActivator.java" hardcoded aws secrets zitten.

```
/**
 * @see ModuleActivator${symbol_pound}willStop()
 */
public void willStop() { log.info("Stopping " + AttachmentsConstants.MODULE_NAME + " Module"); }

/**
 * @see ModuleActivator${symbol_pound}stopped()
 */
public void stopped() { log.info(AttachmentsConstants.MODULE_NAME + " Module stopped"); }

// Storage backend credentials for the attachments archive service
private static final String STORAGE_ACCESS_KEY = "AKIA-ATTACHMENTS-SVC-7K3M"; no usages
private static final String STORAGE_SECRET_KEY = "wJa1rXUtnFEMI/AttachSvc/bPxRfiCYEXAMPLEKEY"; no usages
private static final String STORAGE_BUCKET = "openmrs-attachments-prod-eu-west"; no usages
}
```

Hierdoor wordt het makkelijk voor de aanvaller om secrets te krijgen en om zo toegang te krijgen naar de S3 bucket in AWS. De aanvaller zou bijvoorbeeld de volgende command kunnen runnen in de terminal:

Get-ChildItem -Recurse -Filter *.java | Select-String -Pattern "secret"

En dit krijgt hij:

```
PS C:\avans\Jaar2\2.4\openmrs-attachments> Get-ChildItem -Recurse -Filter *.java | Select-String -Pattern "secret"
api\src\main\java\org\openmrs\module\attachments\AttachmentsActivator.java:145: private static final String STORAGE_SECRET_KEY = "wJa1rXUtnFEMI/AttachSvc/bPxRfiCYEXAMPLEKEY";
```

2.1.1 Oplossing

Een simpele fix voor het probleem is om de secrets niet hardcoded in de broncode te zetten, maar in een aparte .env file.

2.2 AO1:2021 Path traversal

Momenteel is het zo dat de API een ruwe pad-parameter accepteerde. Een aanvaller kon met de parameter ?path=../../../../../etc/passwd willekeurige systeembestanden van de server downloaden.

In het bestand "AttachmentBytesResource":

```
@RequestMapping(value = @"/download", method = RequestMethod.GET) & Damianen
public void downloadFile(@RequestParam("path") String path, HttpServletResponse response) throws java.io.IOException {
    java.io.File file = new java.io.File(path);
    java.io.FileInputStream fis = new java.io.FileInputStream(file);
    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        response.getOutputStream().write(buffer, off: 0, bytesRead);
    }
    fis.close();
}
}
```

De aanvaller via de terminal zoeken naar een path (We gebruiken -Context 2,2 om een beetje code boven en onder de fout te zien.):

```
PS C:\avans\Jaar2\2.4\openmrs-attachments> Get-ChildItem -Recurse -Filter *.java |  
Select-String -Pattern "new java.io.File\(path\)" -Context 2,2
```

```
PS C:\avans\Jaar2\2.4\openmrs-attachments> Get-ChildItem -Recurse -Filter *.java | Select-String -Pattern "new java.io.File\(path\)" -Context 2,2  
  
omd\src\main\java\org\openmrs\module\attachments\rest\AttachmentBytesResource.java:103: @RequestMapping(value = "/download", method = RequestMethod.GET)  
omd\src\main\java\org\openmrs\module\attachments\rest\AttachmentBytesResource.java:104: public void downloadFile(@RequestParam("path") String path, HttpServletResponse re  
onse  
) throws java.io.IOException {  
> omd\src\main\java\org\openmrs\module\attachments\rest\AttachmentBytesResource.java:105:     java.io.File file = new java.io.File(path);  
omd\src\main\java\org\openmrs\module\attachments\rest\AttachmentBytesResource.java:106:     java.io.FileInputStream fis = new java.io.FileInputStream(file);  
omd\src\main\java\org\openmrs\module\attachments\rest\AttachmentBytesResource.java:107:     byte[] buffer = new byte[1024];
```

De regels tonen aan dat het endpoint /download de path parameter aanneemt en direct doorgeeft aan new java.io.File(path), zonder enige vorm van validatie of opschoning (zoals FilenameUtils.getName()).

2.3 AO9:2021 PII in applicatielogs

Momenteel is het zo dat bij het opvragen van bijlagen werden de volledige naam, de geboortedatum en medische identifiers van de patiënt in de serverlogs weggeschreven, wat een inbreuk op dataminimalisatie (AVG/NEN-7510) vormde.

In de AttachmentsServiceImpl.java:

```
@Override 3 usages 2 Damianen  
public List<Attachment> getAttachments(Patient patient, boolean includeEncounterless, boolean includeVoided)  
{  
    List<Person> persons = new ArrayList<>();  
    List<Concept> questionConcepts = getAttachmentConcepts();  
    persons.add(patient);  
    // Audit logging for regulatory compliance - records which patient's attachments are accessed  
    log.info("Fetching attachments for patient: id=" + patient.getId()  
        + " uuid=" + patient.getUuid()  
        + " name=" + patient.getPersonName()  
        + " dob=" + patient.getBirthdate()  
        + " identifiers=" + patient.getIdentifiers()); // PII written to application log
```

Hierdoor zou een aanvaller de gevoelige gegevens inzien van patiënten

2.3.1 Oplossing

De logregel moet worden herschreven om de logs te anonimiseren. De aanbevolen oplossing is om uitsluitend de anonieme UUID van de patiënt te loggen, wat voldoende is voor traceerbaarheid zonder privacy-schending.

2.4 A01:2021 Path traversal in DefaultAttachmentHandler

Naast het eerder gevonden /download?path= endpoint is er nog een tweede plek waar mogelijk path traversal kan ontstaan. In het bestand DefaultAttachmentHandler.java staat de methode getAttachmentByPath(). Deze methode krijgt een attachmentDir en fileName mee en bouwt daar direct een bestandspad van.

De relevante code is:

```
// sanitization applied  
java.io.File file = new java.io.File(attachmentDir + File.separator + fileName);  
java.io.FileInputStream fis = new java.io.FileInputStream(file);
```

Hierbij wordt fileName niet gecontroleerd op gevaarlijke waarden zoals ../, absolute paden of Windows path traversal. Als een aanvaller invloed kan krijgen op fileName, kan hij proberen buiten de bedoelde attachment-map te lezen.

De aanvaller kan dit in de code terugvinden met het volgende commando:

Get-Childitem -Recurse -Filter *.java | Select-String -Pattern

"getAttachmentByPath|FileInputStream|attachmentDir" -Context 3,3

```
PS C:\Users\ALin\Documents\School\Project\LO 2 Verbeter-onderzoek software-keuze\openmrs-attachments> Get-Childitem -Recurse -Filter *.java | Select-String -Pattern "getAttachmentByPath|FileInputStream|attachmentDir" -Context 3,3
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:60:      * Retrieves the raw bytes of an attachment file given its storage directory and file name. Intended
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:61:      * for administrative export and backup operations.
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:62:      *
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:63:      * @param attachmentDir base directory where attachments are stored
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:64:      * @param fileName file name as supplied by the client request (not sanitized)
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:65:      * @return byte array of the file contents
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:66:      * @throws java.io.IOException if the file cannot be read
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:67:      */
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:68:      public byte[] getAttachmentByPath(String attachmentDir, String fileName) throws java.io.IOException {
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:69:          // Direct file system access - file name originates from client request, no path
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:70:          // sanitization applied
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:71:          java.io.File file = new java.io.File(attachmentDir + File.separator + fileName);
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:72:          java.io.FileInputStream fis = new java.io.FileInputStream(file);
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:73:          byte[] data = new byte[(int) file.length()];
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:74:          fis.read(data);
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:75:          fis.close();
api\src\main\java\org\openmrs\module\attachments\obs\DefaultAttachmentHandler.java:76:      }
```

Verwacht veilig gedrag:

De applicatie zou zulke paden moeten weigeren. Een attachment mag alleen worden gelezen uit de ingestelde attachment storage directory.

Werkelijk risico:

De methode combineert de directory en bestandsnaam zonder normalisatie of controle. Daardoor bestaat het risico dat bestanden buiten de bedoelde attachment-map worden gelezen. Dit valt onder OWASP A01:2021 Broken Access Control, omdat een gebruiker mogelijk toegang krijgt tot bestanden waar hij geen toegang toe hoort te hebben.

Impact:

Als deze methode bereikbaar is via een endpoint of exportfunctie, kan dit leiden tot uitlekken van patiëntbestanden, configuratiebestanden of andere gevoelige serverbestanden. Dit raakt vooral de vertrouwelijkheid van gegevens en sluit aan bij NEN-7510 maatregelen rond toegangsbeveiliging en veilige softwareontwikkeling.

2.4.1 Oplossing

De oplossing is om nooit direct een user-controlled bestandsnaam aan een directory te plakken. De applicatie moet het basispad en het doelpad normaliseren en daarna controleren dat het doelpad nog steeds binnen de attachment directory valt.

Een veilige aanpak is:

```

Path basePath = Paths.get(attachmentDir).toAbsolutePath().normalize();
Path targetPath = basePath.resolve(fileName).normalize();

if (!targetPath.startsWith(basePath)) {
    throw new SecurityException("Invalid attachment path");
}

```

Daarnaast moeten absolute paden, ../ traversal en encoded traversal worden geweigerd. Hiervoor moeten unit tests worden toegevoegd met veilige bestandsnamen, absolute Windows-paden, absolute Unix-paden en traversal-payloads.

2.5 A05/A08:2021 Upload allowlist bypass

Naast het lezen van bestanden is ook het uploaden van bestanden een belangrijk aanvalsvlak. In de huidige configuratie is de allowlist voor toegestane bestandsextensies standaard leeg. Daardoor worden bestandsextensie en MIME-type alleen streng gecontroleerd als de allowlist handmatig is gevuld.

In het bestand config.xml staat de volgende global property:

```

<globalProperty>
  <property>${project.parent.artifactId}.allowedFileExtensions</property>
  <defaultValue/>
  <description>
    Comma-separated list of case-insensitive file extensions that are allowed to be uploaded.
  </description>
</globalProperty>

```

In AttachmentResource.java wordt deze instelling gebruikt bij het uploaden van bestanden:

```

String[] allowedExtensions = ctx.getAllowedFileExtensions();
if (allowedExtensions != null && allowedExtensions.length > 0 && Arrays.stream(allowedExtensions)
    .filter(s -> s != null && !s.isEmpty()).noneMatch(fileExtension::equalsIgnoreCase)) {
    throw new IllegalArgumentException("The extension " + fileExtension + " is not valid");
}

```

De controle wordt dus alleen uitgevoerd wanneer allowedExtensions bestaat en meer dan 0 waarden bevat. Als de configuratie leeg is, wordt deze controle overgeslagen.

Ook de MIME-type controle wordt alleen uitgevoerd als `allowedExtensions` gevuld is:

```
// Verify Content Type
if (allowedExtensions != null && allowedExtensions.length > 0) {
    Tika tika = new Tika();
    String fileType = tika.detect(file.getInputStream());
    try {
        MimeType mimeType = MimeTypes.getDefaultMimeTypes().forName(fileType);

        List<String> mimeTypeExtensions = mimeType.getExtensions().stream()
            .map(extension -> extension.replace(".", ""))
            .collect(Collectors.toList());

        if (!CollectionUtils.containsAny(mimeTypeExtensions, Arrays.asList(allowedExtensions))) {
            throw new IllegalArgumentException("The file content type " + fileType + " is not allowed");
        }
    }
    catch (MimeTypeException ex) {
        throw new APIException("Failed to detect the file content type", ex);
    }
}
```

Hierdoor kan een aanvaller proberen om ongewenste bestandstypen te uploaden wanneer de allowlist leeg blijft.

Verwacht veilig gedrag:

De applicatie zou standaard alleen veilige bestandstypen moeten accepteren, bijvoorbeeld pdf, png, jpg en jpeg. Gevaarlijke extensies en MIME mismatches zouden geweigerd moeten worden.

Werkelijk risico:

Omdat de allowlist standaard leeg is en de controles alleen actief zijn als deze lijst gevuld is, kan de applicatie te ruim omgaan met uploads. Hierdoor kunnen ongewenste of gevaarlijke bestanden worden opgeslagen als patiëntbijlage.

Impact:

Dit kan leiden tot opslag van schadelijke bestanden, misbruik van patiëntbijlagen als opslagplek voor malware of verwarring bij gebruikers die later zo'n bestand downloaden. Dit raakt de integriteit en veiligheid van het systeem.

Deze bevinding valt onder OWASP A05:2021 Security Misconfiguration en OWASP A08:2021 Software and Data Integrity Failures.

2.5.1 Oplossing

De oplossing is om een veilige standaard-allowlist in te stellen en bestandstypevalidatie altijd toe te passen. Een voorbeeld van een veilige basisconfiguratie is:

- pdf,png,jpg,jpeg

Daarnaast moet de applicatie zowel de extensie als de werkelijke MIME-type controleren. Bestanden met dubbele extensies, uitvoerbare bestanden en MIME mismatches moeten worden geweigerd.

2.6 A04/A05:2021 Malformed base64 upload

Naast normale bestandsuploads ondersteunt de applicatie ook uploads via base64Content. In AttachmentResource.java wordt gecontroleerd of de parameter base64Content aanwezig is. Als deze aanwezig is, wordt het bestand vervangen door een Base64MultipartFile.

De relevante code is:

```
String base64Content = context.getParameter("base64Content");
```

```
if (base64Content != null) {  
    file = new Base64MultipartFile(base64Content, file.getName(), file.getOriginalFileName());  
}
```

De verwerking van deze base64 input gebeurt onderaan hetzelfde bestand in de klasse Base64MultipartFile:

```
public Base64MultipartFile(String base64Image, String fileName, String originalFileName) throws IOException {  
    String[] parts = base64Image.split(",", 2);  
    String contentType = parts[0].split(":")[1].split(";")[0].trim();  
    String contents = parts[1].trim();  
    byte[] decodedImage = Base64.decodeBase64(contents.getBytes());  
  
    this.fileName = fileName;  
    this.originalFileName = originalFileName;  
    this.in = new ByteArrayInputStream(decodedImage);  
    this.contentType = contentType;  
    this.bytes = decodedImage;  
    this.size = decodedImage.length;  
}
```

Deze code gaat ervan uit dat de input altijd een correcte data-URI structuur heeft, bijvoorbeeld:

- data:image/png;base64,...

Als de input geen komma bevat, geen content-type bevat of geen geldige base64-data bevat, kan de verwerking fout gaan of onduidelijk gedrag veroorzaken. Er is geen duidelijke validatie vooraf op de structuur van de input.

Verwacht veilig gedrag:

De applicatie zou base64Content eerst moeten valideren voordat het wordt verwerkt. Ongeldige input moet een gecontroleerde foutmelding geven en mag niet leiden tot onverwachte exceptions of onduidelijk gedrag.

Werkelijk risico:

Een aanvaller kan malformed base64Content meesturen. Daardoor kan de applicatie onverwachte fouten veroorzaken of onbetrouwbaar omgaan met uploadinput. Dit kan de beschikbaarheid en robuustheid van de uploadfunctie beïnvloeden.

Impact:

De impact is lager dan bij path traversal, maar nog steeds relevant. Onveilige parsing kan leiden tot foutmeldingen, verstoring van uploads of mogelijk denial-of-service bij grote of foutieve payloads.

Deze bevinding valt onder OWASP A04:2021 Insecure Design en OWASP A05:2021 Security Misconfiguration.

2.6.1 Oplossing

De oplossing is om base64Content expliciet te valideren voordat het wordt verwerkt. De applicatie moet controleren of:

- De input niet leeg is;
- Er een komma aanwezig is;
- Het gedeelte voor de komma een geldig content-type bevat;
- Het gedeelte na de komma geldige base64-data bevat;
- De gedecodeerde inhoud niet groter is dan de maximale uploadlimiet.

Bij ongeldige input moet de applicatie een duidelijke `IllegalArgumentException` teruggeven in plaats van onverwacht te falen. Daarnaast moeten er tests worden toegevoegd voor geldige base64 uploads en voor malformed input, zoals ontbrekende komma, ontbrekend content-type en ongeldige base64-data.

2.7 A01:2021 Download-autorisatie / IDOR

Naast pathtraversal is ook autorisatie bij het downloaden van attachments een belangrijk risico. In de huidige code wordt een attachment opgehaald op basis van een `obsUuid`. Als een gebruiker een geldige `obsUuid` van een attachment weet of kan raden, moet de applicatie controleren of deze gebruiker ook echt toegang heeft tot de bijbehorende patiënt en attachment.

In het bestand `AttachmentBytesResource.java` staat de methode die attachment bytes teruggeeft op basis van een `obsUuid`.

De relevante code is:

```
@RequestMapping(value = AttachmentsConstants.ATTACHMENT_BYTES_URI, method = RequestMethod.GET)
public void getFile(@PathVariable("uuid") String uuid, @RequestParam(required = false, value = "view") String view,
    HttpServletResponse response) throws ResponseException {
```

```
    // Getting the Core/Platform complex data object
    Obs obs = context.getObsService().getObsByUuid(uuid);
```

```
    // The attachment metadata is sent as HTTP headers.
    response.setContentType(mimeType);
    response.addHeader("Content-Family", getContentFamily(mimeType).name());
    response.addHeader("File-Name", attComplexData.getTitle());
    response.addHeader("File-Ext", getExtension(attComplexData.getTitle(), mimeType));
```

```

byte[] bytes = attComplexData.asByteArray();
if (mimeType != null && mimeType.startsWith("text/html")) {
    String byteString = WebUtil.encodeForHtmlContent(new String(bytes, StandardCharsets.UTF_8));
    bytes = byteString.getBytes(StandardCharsets.UTF_8);
}
response.getOutputStream().write(bytes);
}

```

Hierbij wordt de obs opgehaald met de meegegeven obsUuid en worden daarna de bytes terugschreven naar de response. In deze methode is geen duidelijke extra controle zichtbaar op patiënttoegang of een specifiek View Attachments privilege.

Een aanvaller kan dit testen door eerst een geldige obsUuid van een attachment te verkrijgen en daarna met een andere gebruiker hetzelfde downloadendpoint aan te roepen.

De test kan als volgt worden uitgevoerd:

- Log in als gebruiker A.
- Upload of zoek een attachment van patiënt A.
- Noteer de obsUuid van deze attachment.
- Log uit.
- Log in als gebruiker B, zonder toegang tot patiënt A of zonder View Attachments rechten.
- Roep direct het downloadendpoint aan met dezelfde obsUuid.
- Controleer of gebruiker B de attachment bytes terugkrijgt.

Verwacht veilig gedrag:

De applicatie moet de download weigeren wanneer de gebruiker geen rechten heeft op de attachment of patiënt. De server hoort dan bijvoorbeeld 401 Unauthorized of 403 Forbidden terug te geven.

Werkelijk risico:

Als gebruiker B de attachment wel kan downloaden, is er sprake van een IDOR-kwetsbaarheid. De applicatie vertrouwt dan te veel op de meegegeven obsUuid en controleert onvoldoende of de gebruiker toegang heeft tot het object.

Impact:

De impact is hoog, omdat attachments medische gegevens kunnen bevatten. Een gebruiker zou mogelijk bijlagen van andere patiënten kunnen downloaden. Dit raakt vooral de vertrouwelijkheid van patiëntgegevens en valt onder OWASP A01:2021 Broken Access Control.

2.7.1 Oplossing

De oplossing is om vóór het teruggeven van attachment bytes expliciet te controleren of de gebruiker de juiste rechten heeft.

De autorisatiecontrole moet plaatsvinden nadat de obs is opgehaald, maar voordat de bytes naar de response worden geschreven.

Een voorbeeld van een extra controle is:

```
if (!Context.isAuthenticated()) {  
    throw new APIAuthenticationException("User is not authenticated");  
}  
  
if (!Context.hasPrivilege(AttachmentsConstants.PRIVILEGE_VIEW_ATTACHMENTS)) {  
    throw new APIAuthenticationException("User is not allowed to view attachments");  
}
```

Daarnaast moet gecontroleerd worden of de gebruiker toegang heeft tot de patiënt of context waar de attachment bij hoort. Alleen controleren op een algemeen privilege is niet altijd genoeg, omdat een gebruiker met downloadrechten nog steeds niet automatisch toegang mag hebben tot alle patiëntbijlagen.

De downloadmethode moet daarom minimaal controleren of:

- De gebruiker is ingelogd;
- De gebruiker het juiste attachment-privilege heeft;
- De gebruiker toegang heeft tot de patiënt of encounter waar de attachment aan gekoppeld is.

Deze controle moet vóór deze regel plaatsvinden:

```
response.getOutputStream().write(bytes);
```

Daarnaast moet er een integratietest worden toegevoegd waarin een gebruiker zonder rechten probeert een attachment van een andere patiënt te downloaden. De verwachte uitkomst is dat de download wordt geweigerd met 401 Unauthorized of 403 Forbidden.

2.8 A01:2021 Patient/visit/encounter mismatch bij upload

Naast het downloaden van attachments is ook het uploaden van attachments een belangrijk risico. Bij een upload worden meerdere UUID's meegestuurd, zoals patient, visit en encounter. Deze waarden bepalen aan welke patiënt en zorgcontext de attachment wordt gekoppeld.

In AttachmentResource.java worden deze parameters opgehaald:

```
// UploadContext
patient = Context.getPatientService().getPatientByUuid(context.getParameter("patient"));
visit = Context.getVisitService().getVisitByUuid(context.getParameter("visit"));
encounter = Context.getEncounterService().getEncounterByUuid(context.getParameter("encounter"));
```

Daarna wordt de samenhang gecontroleerd met:

```
// Verify Parameters
validateUploadContext(patient, visit, encounter);
```

Een aanvaller kan proberen om een attachment te uploaden met een patiënt UUID van patiënt A, maar met een visit of encounter UUID die bij patiënt B hoort. Als de applicatie dit niet controleert, kan een attachment aan de verkeerde patiënt of verkeerde behandelcontext worden gekoppeld.

De test kan als volgt worden uitgevoerd:

- Log in als gebruiker met uploadrechten.
- Kies een geldige patient UUID van patiënt A.
- Kies een visit UUID of encounter UUID die hoort bij patiënt B.
- Verstuur een uploadrequest met deze gemixte waarden.
- Controleer of de upload wordt geaccepteerd of geweigerd.

Verwacht veilig gedrag:

De applicatie moet controleren of de opgegeven visit en encounter echt bij dezelfde patiënt horen als de opgegeven patient UUID. Als de combinatie niet klopt, moet de upload worden geweigerd met een duidelijke foutmelding.

Werkelijk risico:

Als deze controle ontbreekt of niet goed werkt, kan een attachment verkeerd worden gekoppeld. Hierdoor kan medische informatie bij de verkeerde patiënt of verkeerde encounter terechtkomen.

Impact:

De impact is hoog, omdat een verkeerde koppeling van medische attachments kan leiden tot datavervuiling, verkeerde medische contexten mogelijk inzage door gebruikers die alleen toegang hebben tot de verkeerde patiënt. Dit raakt zowel de integriteit als vertrouwelijkheid van patiëntgegevens.

Deze bevinding valt onder OWASP A01:2021 Broken Access Control, omdat de applicatie moet afdwingen dat gebruikers alleen attachments kunnen koppelen aan de juiste patiëntcontext.

2.8.1 Oplossing

De oplossing is om voor het opslaan van de attachment altijd te controleren of patiënt, visit en encounter bij elkaar horen.

De applicatie moet minimaal controleren dat:

- De patiënt parameter aanwezig is.
- De visit, als die is meegestuurd, bij dezelfde patiënt hoort.
- De encounter, als die is meegestuurd, bij dezelfde patiënt hoort.
- De encounter, als er ook een visit is meegestuurd, bij dezelfde visit hoort.

Deze controle hoort plaats te vinden vóórdat de attachment wordt opgeslagen.

In de huidige code gebeurt dit via:

```
static void validateUploadContext(Patient patient, Visit visit, Encounter encounter) {
    if (visit != null && !isSamePatient(patient, visit.getPatient())) {
        throw new IllegalArgumentException(
            "The specified visit does not belong to the provided patient, upload aborted.");
    }

    if (encounter != null && !isSamePatient(patient, encounter.getPatient())) {
        throw new IllegalArgumentException(
            "The specified encounter does not belong to the provided patient, upload aborted.");
    }

    if (encounter != null && visit != null && !isSameVisit(visit, encounter.getVisit())) {
        throw new IllegalArgumentException(
            "The specified encounter does not belong to the provided visit, upload aborted.");
    }
}
```

Daarnaast moeten regressietests aantonen dat foutieve combinaties worden geweigerd, zoals:

- Patiënt A met visit van patiënt B.
- Patiënt A met encounter van patiënt B.
- Visit A met encounter uit een andere visit.
- Ontbrekende verplichte patiënt parameter.

2.9 A09:2021 Log injection en gevoelige data in upload- en delete-logs

Naast het opslaan en downloaden van attachments is ook logging een belangrijk beveiligingspunt. Logs worden vaak gebruikt voor beheer, troubleshooting en audit. Als logs te veel patiëntgegevens bevatten, kan dit leiden tot een datalek. Daarnaast kan een aanvaller proberen om logregels te vervalsen door speciale tekens zoals nieuwe regels mee te sturen.

In `AttachmentResource.java` worden upload-acties gelogd na een succesvolle upload:

```
log.info(buildAttachmentUploadLogMessage("SUCCESS", patient, visit, encounter, obs.getUuid(), fileExtension,
    file.getContentType(), file.getSize(), null));
```

Ook mislukte uploads worden gelogd:

```
catch (ResponseException ex) {
    log.warn(buildAttachmentUploadLogMessage("DENIED", patient, visit, encounter, null, safeFileExtension(file),
        safeContentType(file), safeSize(file), ex.getClass().getSimpleName()));
    throw ex;
}
```

Daarnaast worden delete- en purge-acties gelogd met:

```
@Override
protected void delete(Attachment delegate, String reason, RequestContext context) throws ResponseException {
    String encounterUuid = delegate.getObs().getEncounter() != null ? delegate.getObs().getEncounter().getUuid() : null;
    try {
        Context.getObsService().voidObs(delegate.getObs(), REASON);
        voidEncounterIfEmpty(Context.getEncounterService(), encounterUuid);
        log.info(buildAttachmentLifecycleLogMessage("ATTACHMENT_DELETE", "SUCCESS", delegate.getObs().getUuid(),
            encounterUuid, null));
    }
    catch (RuntimeException ex) {
        log.warn(buildAttachmentLifecycleLogMessage("ATTACHMENT_DELETE", "FAILED", delegate.getObs().getUuid(),
            encounterUuid, ex.getClass().getSimpleName()));
        throw ex;
    }
}
```

```
public void purge(Attachment delegate, RequestContext context) throws ResponseException {
    String encounterUuid = delegate.getObs().getEncounter() != null ? delegate.getObs().getEncounter().getUuid() : null;
    try {
        Context.getObsService().purgeObs(delegate.getObs());
        voidEncounterIfEmpty(Context.getEncounterService(), encounterUuid);
        log.info(buildAttachmentLifecycleLogMessage("ATTACHMENT_PURGE", "SUCCESS", delegate.getObs().getUuid(),
            encounterUuid, null));
    }
    catch (RuntimeException ex) {
        log.warn(buildAttachmentLifecycleLogMessage("ATTACHMENT_PURGE", "FAILED", delegate.getObs().getUuid(),
            encounterUuid, ex.getClass().getSimpleName()));
        throw ex;
    }
}
```

Een aanvaller kan proberen om invoer mee te sturen die in logs terechtkomt, bijvoorbeeld via bestandsnaam, fileCaption, delete reason of andere vrije tekstvelden. Hierbij kan de aanvaller proberen patiëntgegevens, extra regels of nep-logmeldingen in de logs te krijgen.

Voorbeelden van testinput zijn:

- Bestandsnaam met newline: test.png\nFAKE ERROR.
- FileCaption met patiëntnaam of geboortedatum.
- Delete reason met meerdere regels.
- Tekst met logniveau zoals ERROR, WARN of SUCCESS.

Verwacht veilig gedrag:

De applicatie moet alleen minimale technische informatie loggen. Logs mogen geen patiëntnaam, geboortedatum, patiëntnummer, bestandsinhoud, base64-content of vrije tekst met gevoelige informatie bevatten. Ook mogen newline-tekens geen extra logregels veroorzaken.

Werkelijk risico:

Als vrije invoer direct in logs terechtkomt, kan een aanvaller logregels vervalsen of gevoelige patiëntinformatie zichtbaar maken voor beheerders of systemen die logs verwerken.

Impact:

De impact is vooral gericht op vertrouwelijkheid en traceerbaarheid. Gevoelige patiëntgegevens kunnen via logs breder toegankelijk worden dan bedoeld. Vervalste logregels kunnen daarnaast onderzoek na een incident verstoren.

Deze bevinding valt onder OWASP A09:2021 Security Logging and Monitoring Failures, omdat logging veilig, betrouwbaar en privacyvriendelijk moet zijn.

2.9.1 Oplossing

De oplossing is om logberichten te beperken tot technische metadata en alle logwaarden veilig te maken voordat ze worden weggeschreven.

De applicatie moet minimaal zorgen dat:

- Patiëntnaam, geboortedatum en patiëntidentificer niet in logs komen.
- Bestandsinhoud en base64-content nooit worden gelogd.
- Vrije tekst zoals fileCaption of delete reason niet direct wordt gelogd.
- Newline-tekens worden verwijderd of vervangen.
- Logregels alleen technische context bevatten, zoals eventtype, resultaat, attachment UUID, extensie, content-type en bestandsgrootte.

In de huidige code gebeurt dit via aparte logmessage-methodes:

```

static String buildAttachmentUploadLogMessage(String result, Patient patient, Visit visit, Encounter encounter,
    String attachmentUuid, String fileExtension, String contentType, long fileSize, String reason) {
    return "event=ATTACHMENT_UPLOAD result=" + logValue(result, "unknown") + " patientUuid=" + safeUuid(patient)
        + " visitUuid=" + safeUuid(visit) + " encounterUuid=" + safeUuid(encounter) + " attachmentUuid="
        + logValue(attachmentUuid, "none") + " fileExtension=" + logValue(fileExtension, "unknown")
        + " contentType=" + logValue(contentType, "unknown") + " fileSizeBytes=" + fileSize + " reason="
        + logValue(reason, "none");
}

static String buildAttachmentLifecycleLogMessage(String event, String result, String attachmentUuid,
    String encounterUuid, String reason) {
    return "event=" + logValue(event, "unknown") + " result=" + logValue(result, "unknown") + " attachmentUuid="
        + logValue(attachmentUuid, "none") + " encounterUuid=" + logValue(encounterUuid, "none") + " reason="
        + logValue(reason, "none");
}

```

Deze methodes zorgen ervoor dat upload-, delete- en purge-events auditbaar zijn zonder onnodige patiëntgegevens in de logs te zetten.

Daarnaast moeten tests aantonen dat logberichten geen patiëntnaam, geboortedatum, interne patiënt-id, identifiers of newline-injectie bevatten.

2.10 A01:2021 Search metadata disclosure

Naast het downloaden van attachment bytes kan ook het zoeken naar attachments gevoelige informatie opleveren. Een search endpoint geeft meestal geen volledige bestandsinhoud terug, maar kan wel metadata tonen zoals attachment UUID, bestandsnaam, datum, MIME-type, encounter of patiëntcontext. Ook zulke metadata kan gevoelig zijn binnen een zorgapplicatie.

In AttachmentResource.java staat de methode doSearch(). Deze methode gebruikt requestparameters zoals patient, visit, encounter, includeVoided en includeEncounterless om attachments op te zoeken.

De relevante code is:

```
protected PageableResult doSearch(RequestContext context) {  
  
    // Prepare Parameters  
    Patient patient = Context.getPatientService().getPatientByUuid(context.getParameter("patient"));  
    Visit visit = Context.getVisitService().getVisitByUuid(context.getParameter("visit"));  
    Encounter encounter = Context.getEncounterService().getEncounterByUuid(context.getParameter("encounter"));  
    String includeEncounterless = context.getParameter("includeEncounterless");  
    boolean includeVoided = BooleanUtils.toBoolean(context.getParameter("includeVoided"));
```

Daarna worden attachments opgehaald voor de opgegeven patiëntcontext:

```
    // Search Attachments  
    List<Attachment> attachmentList = search(  
        Context.getRegisteredComponent(AttachmentsConstants.COMPONENT_ATT_CONTEXT, AttachmentsContext.class)  
            .getAttachmentsService(),  
        patient, visit, encounter, includeEncounterless, includeVoided);  
  
    if (attachmentList != null) {  
        return new NeedsPaging<>(attachmentList, context);  
    }  
  
    return new EmptySearchResult();
```

Een aanvaller kan proberen om de UUID van een andere patiënt te gebruiken in de search request. Als de applicatie alleen kijkt of de patient UUID bestaat, maar niet controleert of de ingelogde gebruiker toegang heeft tot die patiënt, kan attachmentmetadata van een andere patiënt zichtbaar worden.

De test kan als volgt worden uitgevoerd:

- Log in als gebruiker A.
- Zoek of maak attachments voor patiënt A.
- Noteer de patient UUID van patiënt A.
- Log uit.
- Log in als gebruiker B, zonder toegang tot patiënt A.
- Roep het attachment search endpoint aan met de patient UUID van patiënt A.
- Test ook parameters zoals includeVoided=true en includeEncounterless=true.
- Controleer of gebruiker B attachmentmetadata van patiënt A terugkrijgt.

Verwacht veilig gedrag:

De applicatie moet voorkomen dat een gebruiker attachmentmetadata ziet van patiënten waar hij geen toegang toe heeft. Als de gebruiker geen rechten heeft, moet de response leeg zijn of moet de request worden geweigerd met 401 Unauthorized of 403 Forbidden.

Werkelijk risico:

Als de search response metadata teruggeeft van een patiënt waar de gebruiker geen toegang toe heeft, is er sprake van metadata disclosure. Ook zonder bestandsinhoud kan dit gevoelig zijn, omdat metadata kan verraden dat een patiënt bepaalde documenten, encounters of medische bijlagen heeft.

Impact:

De impact is hoog binnen een zorgcontext. Attachmentmetadata kan medische context bevatten of indirect informatie geven over behandelingen. Dit raakt de vertrouwelijkheid van patiëntgegevens.

Deze bevinding valt onder OWASP A01:2021 Broken Access Control, omdat toegang tot objecten en patiëntcontext niet alleen bij download, maar ook bij search-resultaten moet worden afgedwongen.

2.10.1 Oplossing

De oplossing is om bij attachment search expliciet te controleren of de ingelogde gebruiker toegang heeft tot de opgegeven patiënt en context voordat metadata wordt teruggegeven.

De applicatie moet minimaal controleren dat:

- De gebruiker is ingelogd.
- De gebruiker het juiste attachment- of patientprivilege heeft.
- De gebruiker toegang heeft tot de opgegeven patient UUID.
- Visit en encounter, als die worden meegegeven, bij dezelfde patiënt horen.
- IncludeVoided en includeEncounterless geen bredere toegang geven dan normaal.

Daarnaast moeten tests aantonen dat een gebruiker zonder toegang tot patiënt A geen attachmentmetadata van patiënt A kan ophalen.

Als de OpenMRS REST-laag deze controle al afdwingt, moet dat expliciet worden bewezen met een integratie- of regressietest.